

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ
по лабораторной работе №2
по дисциплине

**МОДЕЛИ РЕШЕНИЯ ЗАДАЧ В
ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ**

Студент гр. 221702
Руководитель

Потоцкий Д.А.
Ивашенко В. П.

Минск 2025

1. Описание лабораторной, её темы и цели

Вариант: 9

Тема: Реализация модели решения задач в интеллектуальных системах.

Цель: Реализовать и исследовать модель решения на ОКМД архитектуре задачи вычисления матрицы значений.

Задание: Получить матрицу значений C , соответствующую размерности pxq .

Дано: Сгенерированные матрицы A, B, E, G заданных размерностей pxm, mxq, lxm, rxq соответственно со значениями в рекомендуемом диапазоне $[-1;1]$.

$$c_{ij} = (\tilde{\wedge} k f_{ijk})(3 * g_{ij} - 2) * g_{ij} + (1 - g_{ij}) * (d_{ij} + (4 * (\tilde{\wedge} k f_{ijk})(\sim) d_{ij}) - 3 * d_{ij}) * g_{ij}$$

$$f_{ijk} = (a_{ik} \rightarrow b_{kj}) * (2 * e_k - 1) * e_k + (b_{kj} \rightarrow a_{ik}) * (1 + (4 * (a_{ik} \rightarrow b_{kj}) - 2) * e_k) * (1 - e_k)$$

$$d_{ij} = (\tilde{\vee} k (a_{ik} \tilde{\wedge} b_{kj}))$$

$$(\tilde{\wedge} k x_k) = (\prod_{k=0}^m x_k)$$

$$(\tilde{\vee} k x_k) = 1 - (\prod_{k=0}^m (1 - x_k))$$

$$x/1 \setminus y = \min(\{x\} \cup \{y\})$$

$$(\sim) = /1 \setminus ; \tilde{\wedge} = /1 \setminus ; x \rightarrow y = \max(\{1 - x\} \cup \{y\})$$

Теоретические сведения: Коэффициент ускорения: $Ky(n, r) = \frac{T_1}{T_n}$, где:

- T_1 – время решения задачи на первой(последовательной) архитектуре
- T_n – время решения задачи на другой (параллельной) архитектуре

Эффективность вычислительных систем: $e(n, r) = \frac{Ky(n, r)}{n}$, где:

- n – количество процессорных элементов в системе (совпадает с количеством этапов конвейера)

Коэффициент расхождения: $D = \frac{L_{sum}}{L_{avg}}$, где:

- L_{sum} — время (суммарная длина программы) решения задачи на (параллельной) архитектуре
- L_{avg} — среднее время (средняя длина программы) решения задачи на той же архитектуре

2. Структура программы

Программа включает в себя следующие модули:

1. **Main** – главный класс, который:
 - Получает входные параметры (m, p, q, n)
 - Проверяет корректность ввода
 - Запускает вычисления через **MatrixCalculator**
 - Выводит результаты (матрицы и параметры)
2. **MatrixCalculator** – выполняет основные вычисления:
 - Создает и заполняет случайными значениями матрицы A, B, E, G
 - Вычисляет матрицу C по заданным формулам
 - Считает временные параметры ($T1, Tn$)
 - Реализует вспомогательные методы для вычисления элементов матрицы C
3. **MatrixOperations** – содержит математические операции:
 - Базовые операции (сложение, умножение, вычитание)
 - Операции нечеткой логики (композиция, Т-норма, импликация)
 - Ведет счетчик вызовов операций
 - Округление чисел
4. **MatrixIO** – отвечает за ввод/вывод:
 - Проверка корректности ввода
 - Вывод матриц в форматированном виде

3. Принцип работы алгоритма

Программа вычисляет матрицу C на основе операций нечеткой логики, используя входные параметры m, p, q, n и случайно сгенерированные матрицы A, B, E, G .

1. **Ввод параметров:**
 - m – число столбцов матрицы A и строк матрицы B
 - p – число строк матрицы A
 - q – число столбцов матрицы B
 - n – количество процессорных элементов
2. **Генерация матриц:**
 - Матрицы $A (p \times m), B (m \times q), E (1 \times m), G (p \times q)$ заполняются случайными числами в диапазоне $[-1, 1]$
3. **Вычисление матрицы $C (p \times q)$ по формуле:**
 - Каждый элемент $C[i][j]$ зависит от A, B, E, G
4. **Анализ временных параметров:**
 - $T1$ – время последовательного выполнения всех операций
 - Tn – время выполнения с учетом параллелизации
 - $Ky = T1 / Tn$
 - $e = Ky / n$
 - $Lavg = Tavg / r$
 - $D = Tn / Lavg$
5. **Вывод результатов:**

- Выводятся матрицы A, B, E, G, C
- Выводятся вычисленные параметры T1, Tn, r, Ky, e, Lsum, Lavg, D

4. Демонстрация результатов работы программы

Время выполнения каждой операции по умолчанию равно 1.

```
m = 5
p = 3
q = 2
n = 1
```

Рисунок 1 – Ввод параметров

```
A:
0,538 -0,755 -0,361 -0,132 0,238
-0,756 -0,547 0,154 -0,560 0,842
-0,081 0,119 -0,168 0,814 -0,537
```

Рисунок 2 – Сгенерированная матрица A

```
B:
0,086 -0,121
0,022 0,705
0,570 -0,047
-0,027 -0,269
0,564 -0,444
```

Рисунок 3 – Сгенерированная матрица B

```
E:
-0,133 -0,833 -0,309 0,124 0,038
```

Рисунок 4 – Сгенерированная матрица E

```
G:
  0,067  -0,405
  0,885   0,131
 -0,270  -0,318
```

Рисунок 5 – Сгенерированная матрица G

```
C:
 -0,809   1,164
 -0,399  -6,089
 -0,661   1,000
```

Рисунок 6 – Итоговая матрица C

```
Parameters:
T1 = 672
Tn = 672
r = 42
Ky = 1.0
e = 1.0
Lsum = 672
Lavg = 17.428571428571427
D = 38.55737704918033
```

Рисунок 7 – Вывод параметров

5. Графики

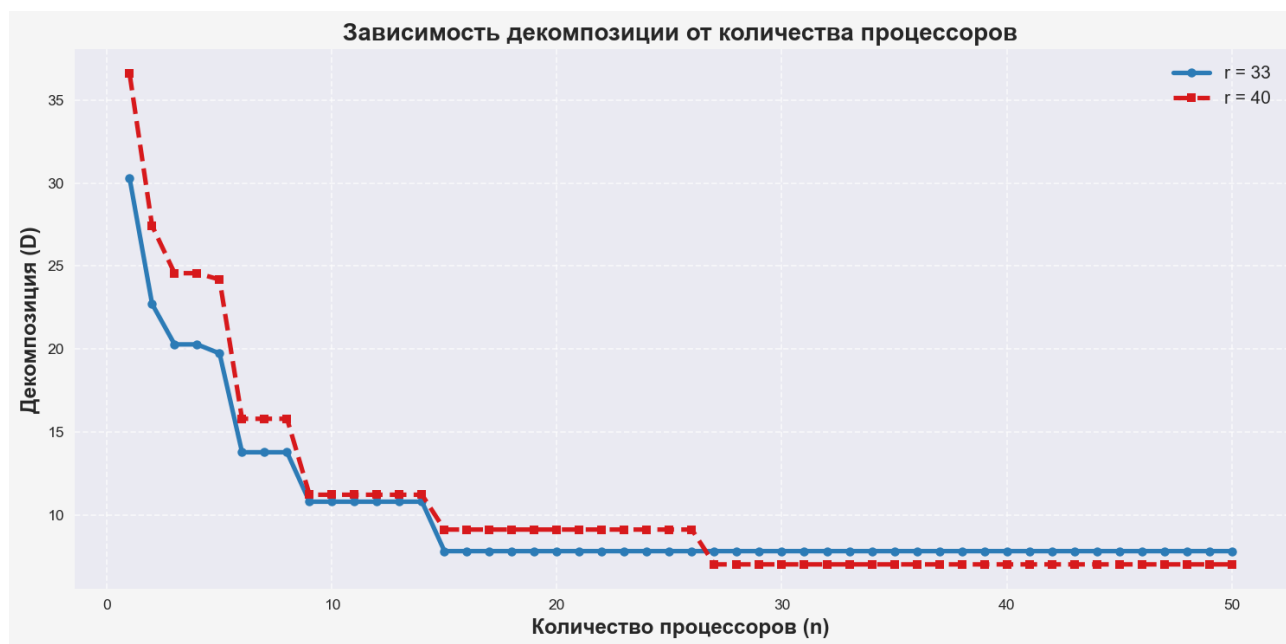


Рисунок 8 – График зависимости коэффициента расхождения от количества процессорных элементов

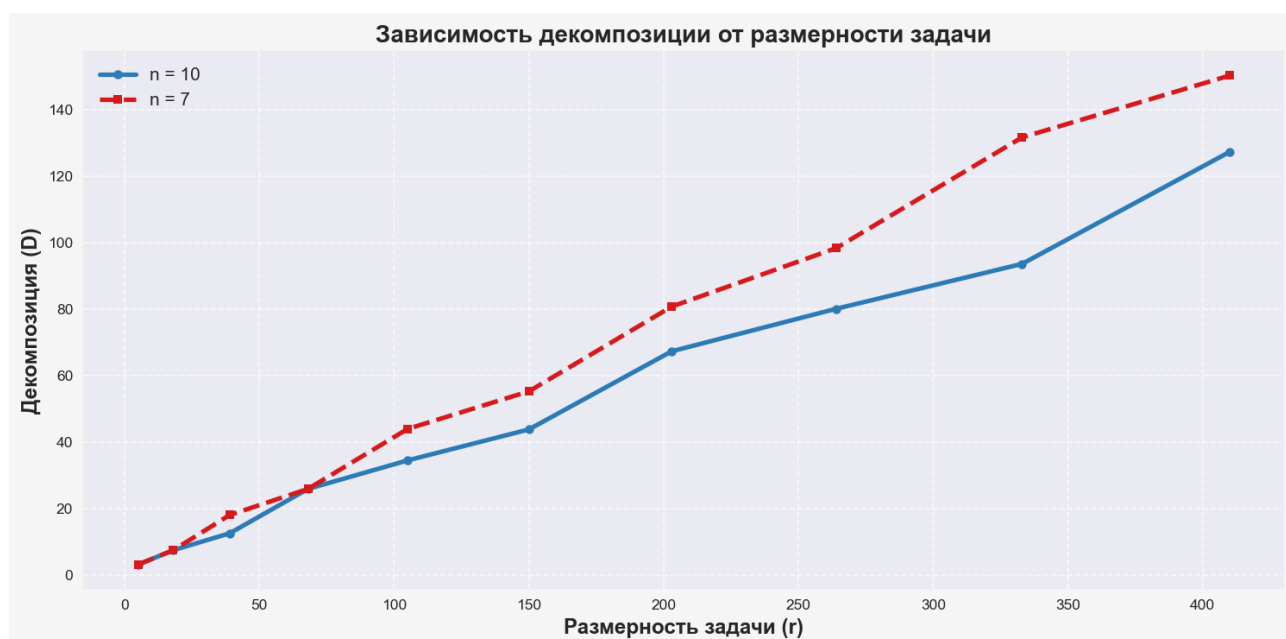


Рисунок 9 – График зависимости коэффициента расхождения от ранга задачи

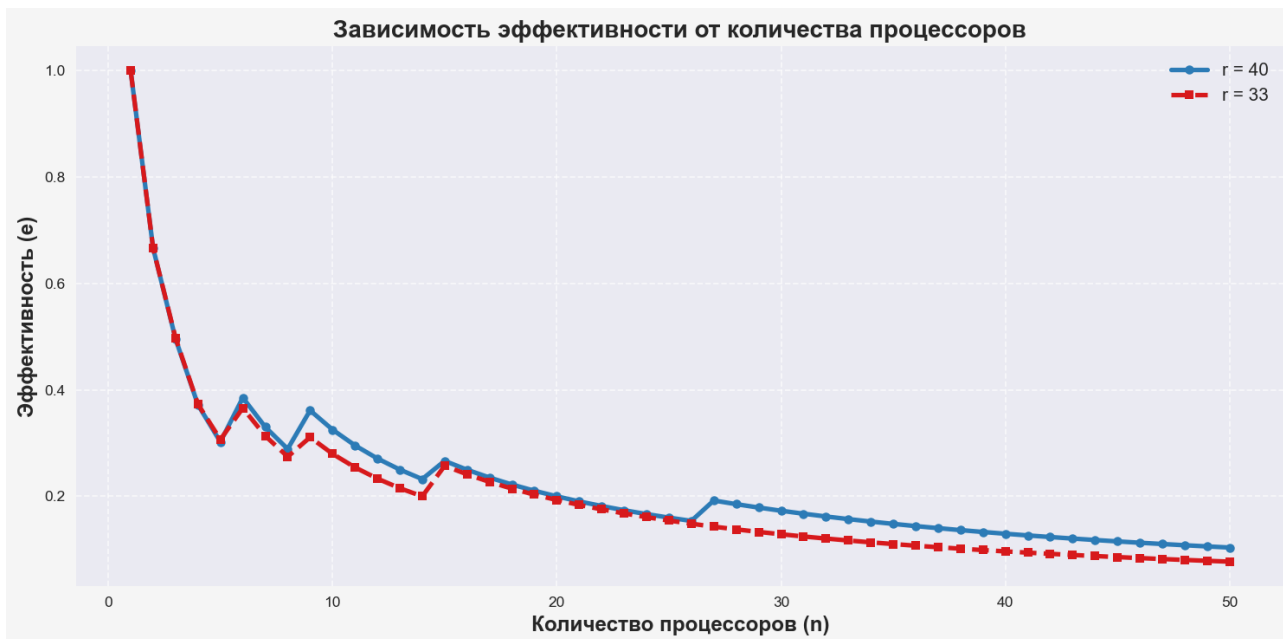


Рисунок 10 – График зависимости эффективности от количества процессорных элементов

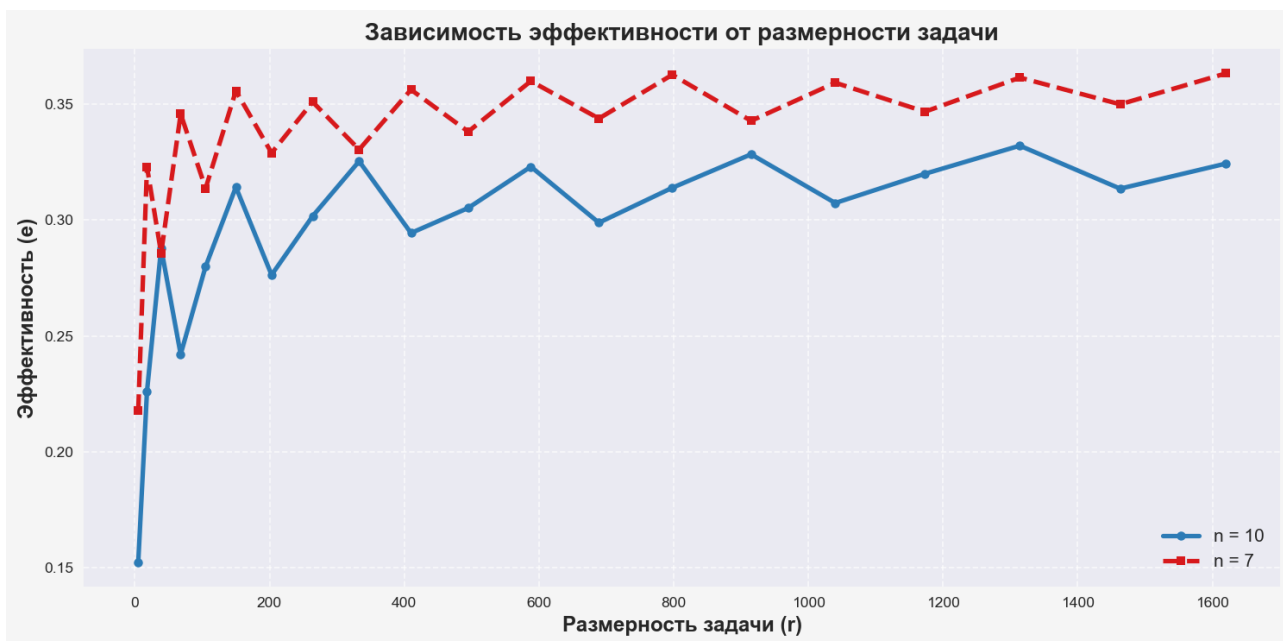


Рисунок 11 – График зависимости эффективности от ранга задачи

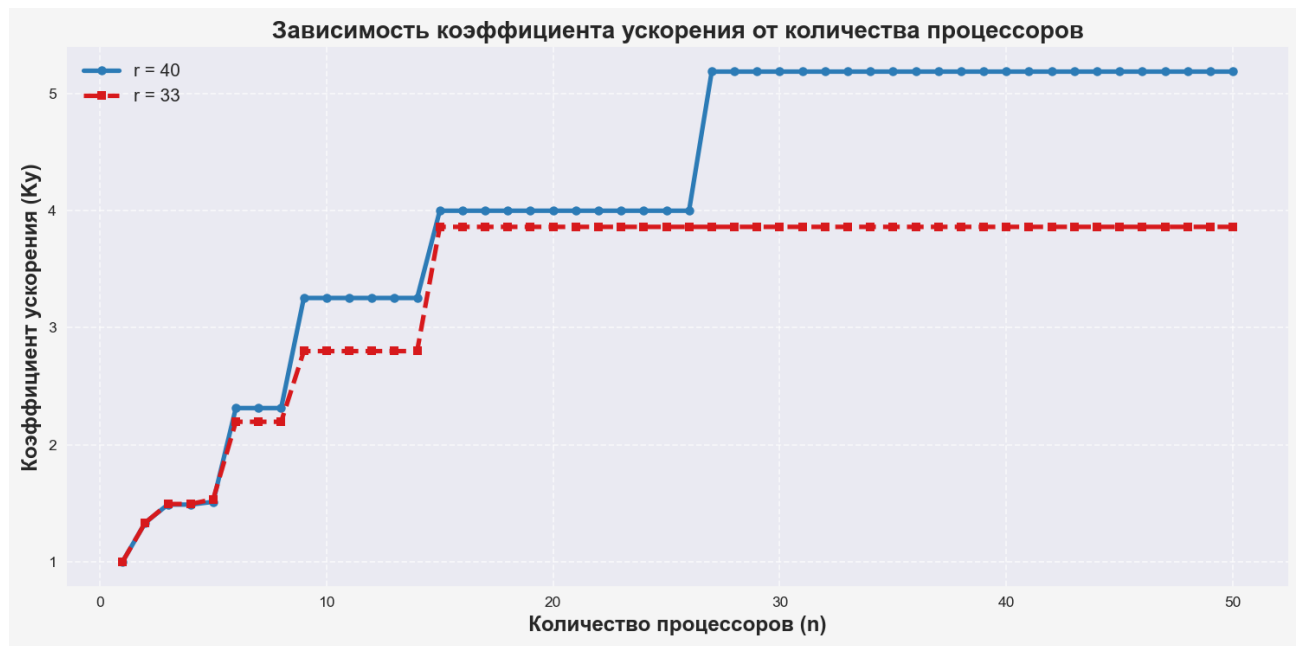


Рисунок 12 – График зависимости коэффициента ускорения от количества процессорных элементов

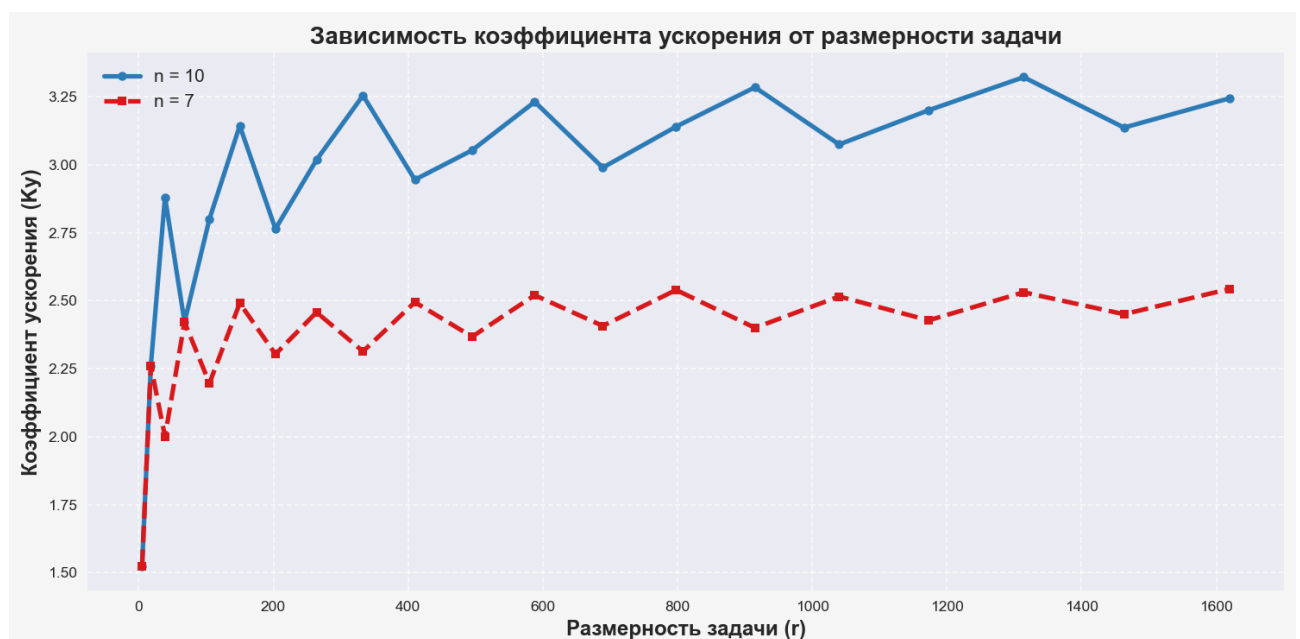


Рисунок 13 – График зависимости коэффициента ускорения от ранга задачи

6. Информационный граф

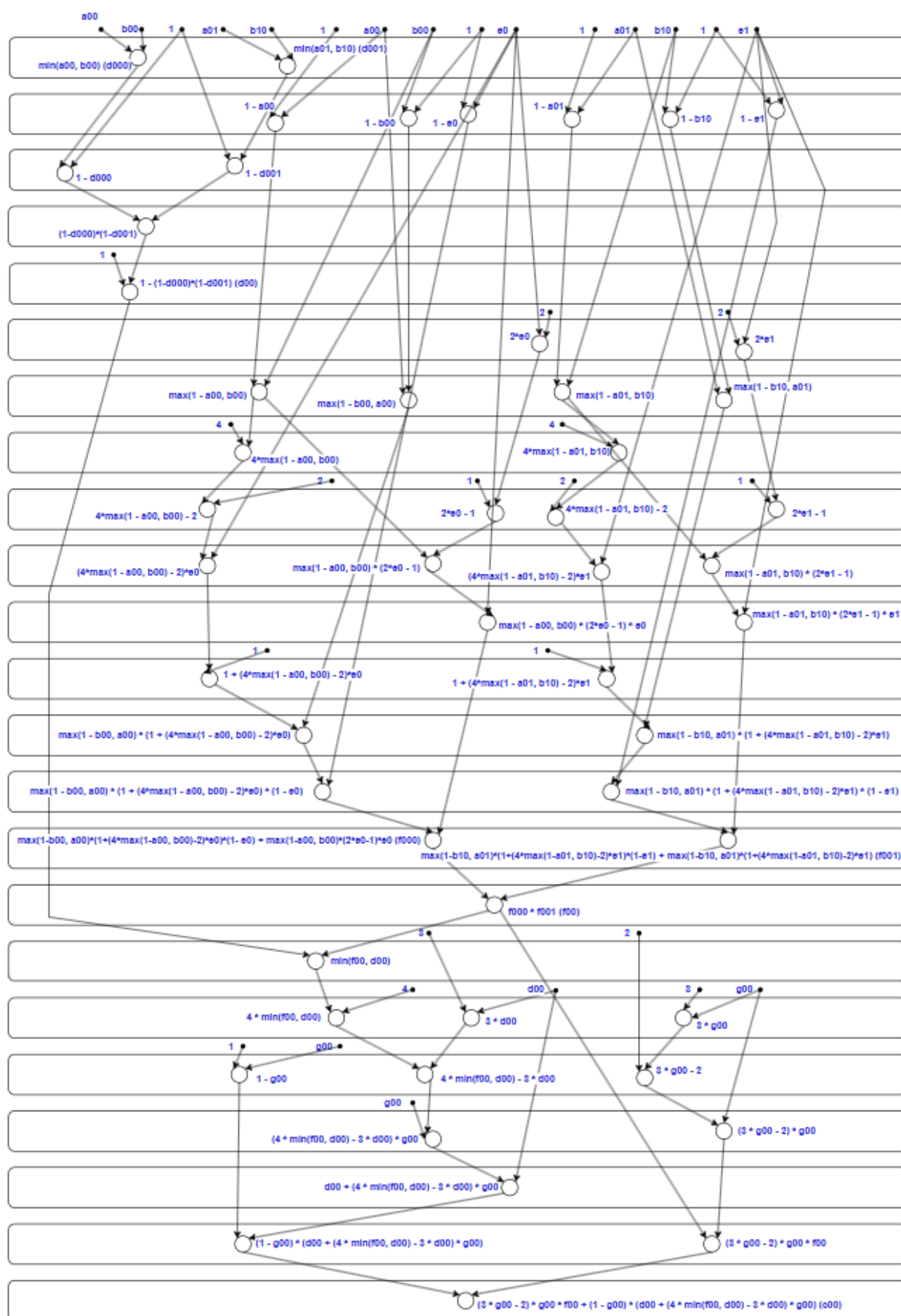


Рисунок 14 – Информационный граф

7. Ответы на контрольные вопросы

1. Проверить, что модель создана верно: программа работает правильно (на всех этапах конвейера).

Модель работает корректно на всех этапах конвейера, что подтверждается поведением графиков функций. График $K_y(r)$ не имеет чёткой асимптоты, но приближается к определённому значению по мере роста аргумента. Рост функции $K_y(r)$ наблюдается на интервалах, где выполняется условие $(m_{i+1} \times 3) \% n - (m_i \times 3) \% n > 0$, а убывание — при обратном соотношении $(m_{i+1} \times 3) \% n - (m_i \times 3) \% n < 0$. После достижения определённого момента график перестаёт изменяться, и это указывает на отсутствие асимптоты графика $K_y(n)$.

Максимум $K_y(n)$ достигается при $n \geq 3 \times m$, где m — аргумент функции $f(p, q, m)$.

Асимптотой графика $e(n)$ является функция $y = 0$.

Аналогичное поведение демонстрирует график функции $e(r)$: при тех же условиях он сначала возрастает, затем убывает, не имея чёткой асимптоты, но приближаясь к ней. Условия роста и убывания аналогичны описанным выше.

График $D(r)$ не имеет чёткой асимптоты, но имеет асимптотическое приближение.

Также и график $D(n)$ лишён точной асимптоты, так как со временем перестаёт изменяться.

Минимальное значение $D(n)$ достигается при $n \geq 3 \times m$, где m снова выступает аргументом в функции $f(p, q, m)$.

Всё это говорит о правильной работе модели и корректности программы на всех этапах обработки.

2. Спрогнозировать, как изменится вид графиков при изменении параметров модели

При увеличении параметра r , график функции $K_y(r)$ будет возрастать неравномерно, с резкими скачками. Если увеличивать n , то $K_y(n)$ также будет расти, но только до тех пор, пока значение n не достигнет трёхкратного значения аргумента m , определяемого функцией $f(p, q, m)$.

Аналогично, при увеличении r функция $e(r)$ тоже будет расти

скачкообразно. Однако функция $e(n)$, напротив, будет уменьшаться при росте n .

График функции $D(r)$ будет демонстрировать рост с увеличением r , тогда как $D(n)$ будет убывать до тех пор, пока n остаётся меньше чем $3 \times m$, где m определяется через функцию $f(p, q, m)$.

3. Каковы ограничения работы архитектуры ОКМД?

Архитектура ОКМД обладает рядом ограничений, связанных как с особенностями организации вычислений, так и с принципами выполнения команд. Одной из ключевых проблем является чувствительность к размеру задачи: если общее количество выполняемых операций меньше количества доступных процессоров, часть ресурсов остаётся незадействованной, что снижает эффективность. Например, при небольших размерах матриц значительная часть процессоров может простаивать. Кроме того, синхронизация между процессорами требует дополнительных затрат времени и может увеличивать общее время выполнения задачи.

Фиксированная структура вычислительного графа также создает ограничения: если на одном из этапов обработки приходится выполнять значительно больше операций, чем на других, возникает «бутылочное горлышко», замедляющее выполнение всей задачи. Даже при оптимальных условиях максимальное ускорение ограничено числом независимых операций, доступных для параллельного исполнения.

Дополнительные трудности вызывает тот факт, что все процессорные элементы архитектуры ОКМД обязаны синхронно выполнять одну и ту же инструкцию. Это особенно неэффективно в ситуациях, когда алгоритм содержит ветвления: если для разных данных требуются разные действия, часть процессоров либо простаивает, либо выполняет лишние операции. Также успешная работа такой архитектуры требует однородных и выровненных данных в памяти. Работа с нерегулярными структурами требует дополнительных преобразований, увеличивающих накладные расходы. Наконец, фиксированная ширина регистров ограничивает обработку массивов, размер которых не кратен этой ширине, что также приводит к неэффективному использованию вычислительных ресурсов.

4. Каковы преимущества работы ОКМД архитектуры перед конвейером?

По сравнению с конвейерной архитектурой, ОКМД демонстрирует лучшие результаты в задачах с высокой степенью параллелизма. В

отличие от конвейера, где параллельность достигается за счёт разделения инструкции на последовательные этапы, ОКМД позволяет выполнять множество однотипных операций над разными данными одновременно, что даёт значительное ускорение вычислений, например, при сложении больших массивов чисел или выполнении матричных операций. Это делает ОКМД особенно эффективной для регулярных задач, таких как обработка изображений или физическое моделирование, где достигается почти линейное ускорение при увеличении числа процессоров, тогда как в конвейере производительность ограничивается зависимостями между инструкциями.

Кроме того, архитектура ОКМД позволяет масштабировать систему, увеличивая количество вычислительных элементов без необходимости перестраивать саму структуру соединений, что упрощает адаптацию под задачи разного масштаба. Благодаря возможности гибкого распределения операций между процессорами система остаётся эффективной даже при изменении параметров задачи, избегая простоев. В отличие от конвейера, где задержка на одном этапе может остановить выполнение всей цепочки, ОКМД менее чувствительна к сбоям и неравномерной загрузке: если один из процессоров завершил работу раньше, он может быть повторно задействован, что повышает утилизацию ресурсов. Кроме того, управление потоками в ОКМД требует меньших накладных расходов, чем синхронизация этапов в конвейере, что также положительно влияет на общую производительность.

5. Каково возможное и реализованное время вычисления редукции массива на архитектуре ОКМД?

Редукция массива на архитектуре ОКМД (SIMD) в теории выполняется быстро благодаря параллельной обработке данных. При таком подходе элементы объединяются попарно на каждом этапе, сокращая общий объём данных вдвое. В результате общее число шагов составляет $\log_2(N)$, где N — число элементов в массиве. К примеру, если массив содержит 256 элементов, потребуется 8 шагов, чтобы получить итоговый результат.

На практике же время выполнения зависит от таких факторов, как количество доступных процессоров, накладные расходы на синхронизацию и передачу данных. Эффективность оценивается выражением $O(N/P + \log P)$, где P — число процессоров. Если число процессоров существенно меньше числа элементов, массив разбивается на блоки размером N/P , обрабатываемые последовательно, а затем результаты этих блоков сводятся за $\log_2(P)$ шагов.

Альтернативный подход предполагает, что при m арифметических операциях и n процессорах теоретическое время редукции составляет $\lceil m/n \rceil$ тактов при одном такте на операцию. Например, если $m = 5$ и $n = 2$, редукция завершается за 3 такта. Однако на практике дополнительные затраты времени вызывают отклонения от теоретических предсказаний. При избыточном числе процессоров большая часть времени тратится на управление и синхронизацию. Для улучшения производительности используются схемы бинарной редукции, позволяющие выполнять операции за $\log_2(m)$ шагов, но они требуют более сложного управления распределением задач.

6. Что показывает коэффициент расхождения и как добиться уменьшения его значения?

Коэффициент расхождения D отражает, насколько равномерно распределена нагрузка между процессорами в параллельной системе. Он рассчитывается как отношение суммарной загрузки к средней, и его минимальное значение — единица, что свидетельствует о полной сбалансированности. Повышение D указывает на то, что некоторые процессоры недогружены, в то время как другие перегружены или работают дольше. Такая ситуация часто возникает при избыточном количестве процессоров или при неравномерной сложности операций. Снизить значение D можно несколькими способами: применением динамического назначения задач в реальном времени, оптимальным выбором числа процессоров (близким к $\sqrt{3m}$), а также разбивкой сложных этапов на более мелкие, чтобы выровнять нагрузку. В хорошо сбалансированных задачах, например при $m=4$ и $n=12$, когда каждое из 12 действий выполняется отдельным процессором, коэффициент D приближается к 1, указывая на равномерное распределение вычислительных усилий.

8. Вывод

В ходе выполнения лабораторной работы была разработана и исследована модель решения задачи вычисления матрицы значений на архитектуре ОКМД. Проведены отладка и тестирование модели, а также построен информационный граф. Построены и проанализированы графики трёх ключевых характеристик конвейерной архитектуры: коэффициента ускорения, эффективности и расхождения.

Список использованных источников

[1] Ивашенко В. П. Модели решения задач в интеллектуальных системах. В 2 ч. Ч. 1 : М74 Формальные модели обработки информации и параллельные модели решения задач : учеб.-метод. пособие / В. П. Ивашенко. — 2020.